https://doi.org/10.0000/tmsce.2026.042

On the Convergence of Fourier Partial Sums

J. Smith[1] and A. Jones[2]

[1]Department of Mathematics, State University, USA
[2]Department of Applied Sciences, Tech Institute, UK
Corresponding author: j.smith@stateuniv.edu

Abstract. We present a numerical investigation of the convergence behavior of Fourier partial sums for discontinuous periodic functions. Using computational methods, we visualize the Gibbs phenomenon and quantify the overshoot at jump discontinuities. Our analysis confirms the classical 9% overshoot bound and demonstrates the utility of literate programming for reproducible mathematical exposition.

Key words: Fourier series; Gibbs phenomenon; convergence; literate programming

Received: 15 January 2026 **Revised:** 10 February 2026 **Accepted:** 20 February 2026

1. Introduction

The Fourier series provides a decomposition of periodic functions into sinusoidal components [**?**]. For smooth functions, the partial sums converge uniformly. For functions with jump discontinuities, convergence is pointwise but not uniform, and the partial sums exhibit characteristic overshoot near the jumps.

This phenomenon, first described by Gibbs, produces an overshoot of approximately 9% of the jump magnitude regardless of the number of terms retained.

We demonstrate these properties computationally using Inkwell's runnable code blocks, combining mathematical exposition with reproducible analysis [1].

2. Fourier Partial Sums

The partial sum of a square wave's Fourier series is

$$f_n(x) = \sum_{k=1}^{n} \frac{4}{(2k-1)\pi} \sin\big((2k-1)x\big). \tag{1}$$

As $n \to \infty$, $f_n(x) \to f(x)$ pointwise for all x not at a discontinuity.

{python file=".inkwell/scripts/sine_plot.py" output="sine_plot" caption="Fourier partial sums for n = 1, 3, 5, 9 showing convergence and Gibbs overshoot."}

3. Regression Example

We also demonstrate Inkwell's inline code execution with a simple regression:

“‘{python display=“both” output=“scatter” caption=“Simulated bivariate data with ordinary least squares fit.”} import os, numpy as np import matplotlib; matplotlib.use(“Agg”) import matplotlib.pyplot as plt

rng = np.random.default_rng(42) x = rng.normal(0, 1, 150) y = 0.7 * x + rng.normal(0, 0.35, 150)

fig, ax = plt.subplots(figsize=(5, 3.5)) ax.scatter(x, y, s=14, alpha=0.6, color=“#4A90D9”) m, b = np.polyfit(x, y, 1) xs = np.sort(x) ax.plot(xs, m * xs + b, color=“#E74C3C”, linewidth=1.5, label=f”$y = m : .2f x' +' if b >= 0 else '' b : .2f$“) ax.set_xlabel(”$x$“); ax.set_ylabel(”$y$“) ax.legend(); ax.grid(alpha=0.2) fig.tight_layout() fig.savefig(os.path.join(os.environ.get(”INKWELL_OUTPUT “.”), “scatter.png”), dpi=200, bbox_inches=“tight”) plt.close(fig) print(f”n = {len(x)}, r = {np.corrcoef(x, y)[0,1]:.3f}“) “‘

4. Conclusion

The computational examples above confirm the classical convergence properties of Fourier series and demonstrate that Inkwell’s code block system produces reproducible, publication-ready output through the TMSCE journal template.

References

[1] D. E. Knuth. Literate programming. *The Computer Journal*, **27** pp. 97–111, 1984. doi:10.1093/comjnl/27.2.97.